



中山大學

SUN YAT-SEN UNIVERSITY

移动机器人规划与控制课程

期末报告

姓 名 _____ 杨皓翔

学 号 _____ 23354179

学 院 _____ 智能工程学院

专 业 _____ 智能科学与技术

2026 年 1 月 18 日

目录

1 Homework 部分	1
1.1 题目 1: 坐标系转换 (末端执行器世界系姿态)	1
1.1.1 题目描述	1
1.1.2 解题思路	1
1.1.3 结果展示	2
1.2 题目 2: 开放题 (规划 / 控制 / 动力学)	2
1.2.1 A* 轨迹规划的启发式与路径简化	2
1.2.2 SO(3) 位置控制器的力姿态生成	3
1.2.3 动力学建模与约束	4
1.3 题目 3 (附加题): 无人机微分平坦性	5
1.3.1 题目描述	5
1.3.2 解题思路	5
1.3.3 结果展示	6
2 Code 部分	7
2.1 任务一: 补全四旋翼动力学模型	7
2.1.1 题目描述	7
2.1.2 补全思路	7
2.1.3 结果展示	8
2.2 任务二: 补充前端规划模块 (A* 搜索)	8
2.2.1 题目描述	8
2.2.2 补全思路	9
2.2.3 结果展示	10
2.3 任务三: 控制器参数调试与评估	11
2.3.1 题目描述	11
2.3.2 调参思路与初步结果	11
2.4 任务四 (附加题): 有效改进与对比实验	12
2.4.1 题目描述	12
2.4.2 改进思路与对比实验	12
2.4.3 最终结果展示	15

1 Homework 部分

1.1 题目 1：坐标系转换（末端执行器世界系姿态）

1.1.1 题目描述

- 给定无人机在飞行过程中的姿态序列（由 `tracking.csv` 提供，以四元数表示无人机机体系 $\{B\}$ 相对世界系 $\{W\}$ 的姿态），以及末端执行器固连坐标系 $\{D\}$ 相对机体系 $\{B\}$ 的时变姿态关系 ${}^B\mathbf{R}_D(t)$ （题目已给出矩阵表达式与参数）。
- 任务要求：计算末端执行器在世界坐标系下的姿态 ${}^W\mathbf{R}_D(t)$ 并转换为四元数 $\mathbf{q}_{WD}(t)$ ；绘制四元数分量随时间变化曲线。
- 输出规范：最终输出前需确保四元数归一化，并满足 $q_w \geq 0$ ；若出现 $q_w < 0$ ，则将四元数整体取反以保证时间序列连续性；曲线图要求刻度清晰可辨。

1.1.2 解题思路

本题本质是“姿态链式变换”。无人机机体系 $\{B\}$ 在世界系 $\{W\}$ 中的姿态已知（由 $\mathbf{q}_{WB}(t)$ 给出），末端执行器坐标系 $\{D\}$ 相对机体系 $\{B\}$ 的姿态 ${}^B\mathbf{R}_D(t)$ 也已知，因此末端在世界系下的姿态可以通过两次旋转复合得到。

具体计算流程如下：

1. **读取并规范化无人机姿态**：从 `tracking.csv` 读取每个时刻的四元数 $\mathbf{q}_{WB}(t)$ ，首先对其进行归一化，以避免数值误差导致的非单位四元数影响旋转矩阵的正交性。随后将 $\mathbf{q}_{WB}(t)$ 转换为旋转矩阵 ${}^W\mathbf{R}_B(t)$ 。
2. **构造末端相对机体的旋转**：根据题目给出的解析表达式，利用参数 $\omega = 0.5 \text{ rad/s}$ 、 $\alpha = \pi/12$ ，在每个时间点计算 ${}^B\mathbf{R}_D(t)$ ，该矩阵描述末端在机体系内的时变姿态运动。
3. **姿态复合得到世界系末端姿态**：通过旋转矩阵乘法完成坐标系链路变换：

$${}^W\mathbf{R}_D(t) = {}^W\mathbf{R}_B(t) {}^B\mathbf{R}_D(t). \quad (1)$$

该式表示先从 $\{D\}$ 旋到 $\{B\}$ ，再从 $\{B\}$ 旋到 $\{W\}$ ，得到末端在世界系的最终姿态。

4. **转换回四元数并处理连续性**：将 ${}^W\mathbf{R}_D(t)$ 转换为四元数 $\mathbf{q}_{WD}(t)$ 并归一化。由于四元数 $\mathbf{q}$ 与 $-\mathbf{q}$ 表示同一旋转，为避免时间序列中出现符号翻转造成的曲线跳变，对相邻时刻做连续性处理：若当前四元数与上一时刻四元数点积小于 0，则将当前四元数整体取反，从而保证曲线连续平滑。
5. **绘图与输出**：绘制 $\mathbf{q}_{WD}(t)$ 的四个分量随时间变化曲线并保存，以便在报告中展示。

1.1.3 结果展示

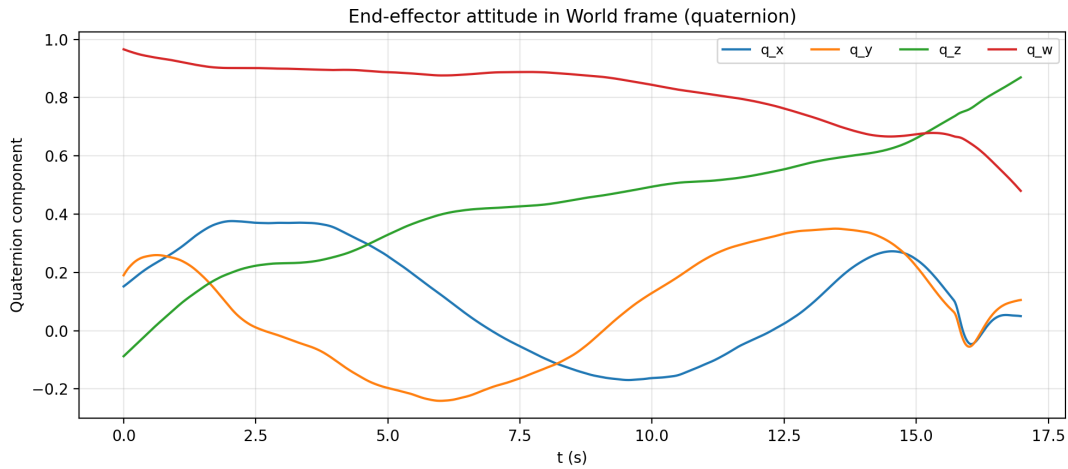


图 1: Homework 题目 1

1.2 题目 2: 开放题 (规划 / 控制 / 动力学)

1.2.1 A* 轨迹规划的启发式与路径简化

(1) tie_breaker 对开集拓展顺序与路径平滑性的影响

在 Astar_searcher.cpp 中, 启发式为欧氏距离并乘以 $1 + 10^{-5}$:

$$h(\mathbf{x}) = (1 + 10^{-5})\|\mathbf{x} - \mathbf{x}_{goal}\|_2, \quad f(\mathbf{x}) = g(\mathbf{x}) + h(\mathbf{x}).$$

其影响为:

- **扩展顺序更偏向目标:** 在 g 相近时, h 被轻微放大使得更靠近目标的节点更早被弹出扩展, 减少在并列 f 值时的随机扩展。
- **路径更少走之字:** 由于并列减少, 搜索更倾向沿目标方向推进, 通常会减少锯齿式绕行, 路径观感更直、更平滑。

(2) 若想更偏好平面飞行 (减少高度变化), 如何修改启发或边权以及对可行性和最优性的影响

AstarGetSucc 使用 26 邻域扩展, 当前边权为三维步长欧氏距离 $c = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$ 。若希望更偏好平面飞行, 一种直接方法是对高度变化加权, 例如将边权改为

$$c = \sqrt{\Delta x^2 + \Delta y^2 + \lambda_z^2 \Delta z^2}, \quad \lambda_z > 1,$$

并将启发式也改为同一加权距离

$$h = \sqrt{(x - x_g)^2 + (y - y_g)^2 + \lambda_z^2 (z - z_g)^2}.$$

这样会显式提高 $\Delta z \neq 0$ 的代价，促使 A^* 更倾向在 xy 平面内绕行、减少不必要的爬升/下降。

- **可行性**：避障与搜索空间不变，算法仍能找到可行路径；但若必须通过高度变化才能绕过障碍，过大的 λ_z 会使路径更长、甚至更难搜索到（被迫在平面内绕很大圈）。
- **最优性**：若 g 与 h 采用同一加权度量，可保证在该“加权代价”意义下的最优；但它不再是原始欧氏边权意义下的最短路。

(3) path_resolution 过大/过小对跟踪误差与安全性的影响

pathSimplify 采用 Douglas–Peucker 递归简化：当中间点到首尾连线的最大垂距 $d_{\max} \leq \text{path_resolution}$ 时，用首尾点替代该段，从而减少拐点。结合四旋翼的动态约束（可实现的加速度与姿态倾角有限），有：

- **path_resolution 过大**：路径点被大量删减，折线段更长、转向更“突然”，后端拟合时可能在狭窄障碍间更贴边，安全裕度变小；同时长段直连会对动态可行性提出更高要求（需要更大横向加速度/倾角才能跟上），容易在复杂区域产生跟踪误差或触发碰撞风险。
- **path_resolution 过小**：路径点过密会保留栅格噪声（特别是 z 方向的离散起伏），导致后端轨迹在短距离内频繁拐弯/上下波动，控制器需要频繁改变姿态与加速度，容易出现抖动与 RMSE 上升；同时短段时间分配不足时，跟踪会明显滞后。

1.2.2 SO(3) 位置控制器的力姿态生成

(1) calculateControl 中各项在飞行中的作用

在 S03Control.cpp 的 calculateControl 中，总力指令由多项叠加得到：

- **位置反馈 $K_x(\mathbf{p}_d - \mathbf{p})$** ：提供“刚度”，位置误差越大，纠正力越大。
- **速度反馈 $K_v(\mathbf{v}_d - \mathbf{v})$** ：提供“阻尼”，抑制振荡与超调，提高收敛速度。
- **加速度前馈 $m\mathbf{a}_d$** ：在加速/转弯等动态段减少滞后，改善轨迹跟随。
- **重力补偿 $m\mathbf{g}$** ：保证悬停/匀速时无需靠误差产生推力，减小稳态偏差。
- **与测得加速度相关的项 $mK_a(\mathbf{a}_d - \mathbf{a})$** ：用于补偿模型误差和外扰（如阻力、推力建模误差），让实际加速度更快贴近期望。

控制器随后用总力方向生成姿态：将机体系 z 轴对齐 $\mathbf{f}$ 的方向，并结合期望偏航构造旋转矩阵/四元数；同时通过阈值限制 $\mathbf{f}$ 与竖直方向夹角不超过设定值，以避免过大倾角导致失稳或高度掉落。

(2) 为什么需要对 k_a 截断或限幅

代码中 k_a 随误差变化并带截断（误差过大时置零），其原因是：

- **噪声敏感：**IMU 加速度测量含高频噪声，若 K_a 在大范围内保持较大，会把噪声直接映射为推力抖动，引起姿态抖动甚至振荡。
- **大误差阶段更易饱和：**大误差时往往已接近推力/姿态极限，继续依赖加速度修正可能产生过激指令，导致超调、振荡或不稳定，因此需要在大误差时削弱该项。

(3) 更重机体下如何同时调整质量参数与 k_x/k_v

若机体质量增大但希望保持相似的加速度跟踪效果，可以这么调整：

- 先将控制器中的质量参数 m 设为真实值，使重力补偿与前馈项 $m\mathbf{a}_d$ 的幅值正确；
- 再将 k_x, k_v 适当增大（通常可近似按质量同比例放大），因为反馈项本质产生的是力，而加速度为 $\mathbf{f}/m$ 。若不增大增益，闭环响应会变“钝”、误差变大；若增益过大，则推力与倾角需求上升，可能触发饱和并降低稳定裕度。

1.2.3 动力学建模与约束

(1) 建模简化如何影响控制分配？质量/阻尼估计有误差会出现什么现象？

仿真器使用四旋翼刚体模型：推力/力矩由电机转速产生，姿态动力学由惯量矩阵决定，并加入简化阻力项。质量、惯量与阻尼的简化会影响“控制输入 $\rightarrow$ 运动响应”的映射，从而影响控制分配与轨迹可跟踪性：

- **质量估计误差：**低估质量常表现为推力不足、加速段/爬升段滞后与稳态偏差；高估质量则可能推力偏大、出现超调与振荡倾向。
- **惯量估计误差：**惯量不匹配会导致姿态响应过快/过慢，表现为姿态超调抖动或转向迟缓。
- **阻尼估计误差：**阻力低估时高速段“刹不住”易超调，RMSE 增大；阻力高估时速度上不去，轨迹执行更拖沓（时间变长）。

(2) 若加入简单气动阻力，应在控制层还是规划层调整，给出调参思路

若加入简单气动阻力模型，可在两处处理，但各有优劣：

- **控制层 (SO3Control) 补偿：**在力指令中加入阻力补偿项 $-\mathbf{F}_d$ 。优点是能在不显著降低飞行速度的情况下改善高速段跟踪；缺点是依赖 k_d 估计且对速度噪声敏感，需要滤波/限幅。
- **规划层 (轨迹限速/时间分配)：**通过降低最大速度/加速度或增大时间分配，避免进入阻力显著区间。优点是简单鲁棒、安全性好；缺点是总时间变长，综合评分中的 time 项可能变差。

调参思路：若选择控制层补偿，可从较小的 k_d 开始逐步增大，重点观察高速段的系统性滞后是否下降、RMSE 是否改善，同时检查是否引入推力抖动；必要时对速度测量加低通滤波并对补偿项限幅，以在“改善跟踪”和“避免抖动”之间取得平衡。

1.3 题目 3（附加题）：无人机微分平坦性

1.3.1 题目描述

- 本题为选做附加题，利用四旋翼的**微分平坦性**性质（不考虑空气阻力），由给定的世界系轨迹位置 $\mathbf{p}(t)$ 与偏航角 $\psi(t)$ 代数地推导并计算无人机在整个飞行过程中的姿态。
- 题设给定：无人机轨迹为世界坐标系下的**水平面双纽线**；并规定偏航角 $\psi(t)$ 始终与速度方向对齐；机体采用 **FLU**（前-左-上）坐标系定义。
- 任务要求：
 - 推导过程：由 $\mathbf{p}(t)$ 、 $\psi(t)$ 推导机体三轴在世界系中的方向向量，据此构造世界系到机体的旋转矩阵，并转换为四元数。
 - 编程实现：在 `code/src` 下创建新的 ROS 包 `quadrotor_df`，使用 C++ 与 Eigen 完成向量/矩阵计算；在指定时间区间内以合适步长（例如 0.02s）离散采样，计算每个时刻姿态四元数。
 - 输出文件：将结果保存为 `df_quaternion.csv`（时间保留两位小数、四元数保留 7 位小数），并满足四元数归一化与 $q_w \geq 0$ ；绘制四元数变化曲线并放入报告。

1.3.2 解题思路

本题利用四旋翼的微分平坦性：系统姿态可以由平坦输出（位置与偏航角）及其导数代数构造得到。已知轨迹 $\mathbf{p}(t) = [x(t), y(t), z(t)]^\top$ ，并规定偏航角始终与速度方向对齐，因此首先由

$$\dot{\mathbf{p}}(t) = [\dot{x}(t), \dot{y}(t), \dot{z}(t)]^\top, \quad \psi(t) = \text{atan2}(\dot{y}(t), \dot{x}(t))$$

确定每个时刻的偏航方向。

在忽略空气阻力时，四旋翼的合力方向由期望加速度与重力共同决定。令

$$\mathbf{f}(t) = \ddot{\mathbf{p}}(t) + g\mathbf{e}_3, \quad \mathbf{e}_3 = [0, 0, 1]^\top,$$

则机体系的 z 轴（ $\mathbf{U}_p$ ）在世界系中的方向为

$$\mathbf{b}_z(t) = \frac{\mathbf{f}(t)}{\|\mathbf{f}(t)\|}.$$

同时根据 FLU 坐标系定义，机体系 x 轴（Front）在水平面内指向偏航方向：

$$\mathbf{b}_x^{\text{yaw}}(t) = [\cos \psi(t), \sin \psi(t), 0]^\top.$$

为构造满足正交的机体系三轴，先由叉乘得到

$$\mathbf{b}_y(t) = \frac{\mathbf{b}_z(t) \times \mathbf{b}_x^{\text{yaw}}(t)}{\|\mathbf{b}_z(t) \times \mathbf{b}_x^{\text{yaw}}(t)\|},$$

再重新正交化得到

$$\mathbf{b}_x(t) = \mathbf{b}_y(t) \times \mathbf{b}_z(t).$$

于是世界系到机体系的姿态可由旋转矩阵表示为

$${}^W\mathbf{R}_B(t) = \begin{bmatrix} \mathbf{b}_x(t) & \mathbf{b}_y(t) & \mathbf{b}_z(t) \end{bmatrix},$$

并将其转换为四元数 $\mathbf{q}_{WB}(t)$ 输出。

数值实现中，对 $t \in [0, 2\pi]$ 以固定步长（如 0.02 s）离散采样；由离散点计算位置、速度与加速度（可采用数值微分），进而得到上述姿态构造所需的 $\psi(t)$ 与 $\mathbf{b}_z(t)$ 。每一步对四元数进行归一化，并通过相邻时刻四元数点积判断符号连续性：若与上一帧点积小于 0，则整体取反，以消除 $\mathbf{q}$ 与 $-\mathbf{q}$ 等价带来的曲线跳变；同时强制 $q_w \geq 0$ 满足输出要求。最终将结果写入 `df_quaternion.csv`，并据此绘制四元数分量随时间变化曲线用于报告展示。

1.3.3 结果展示

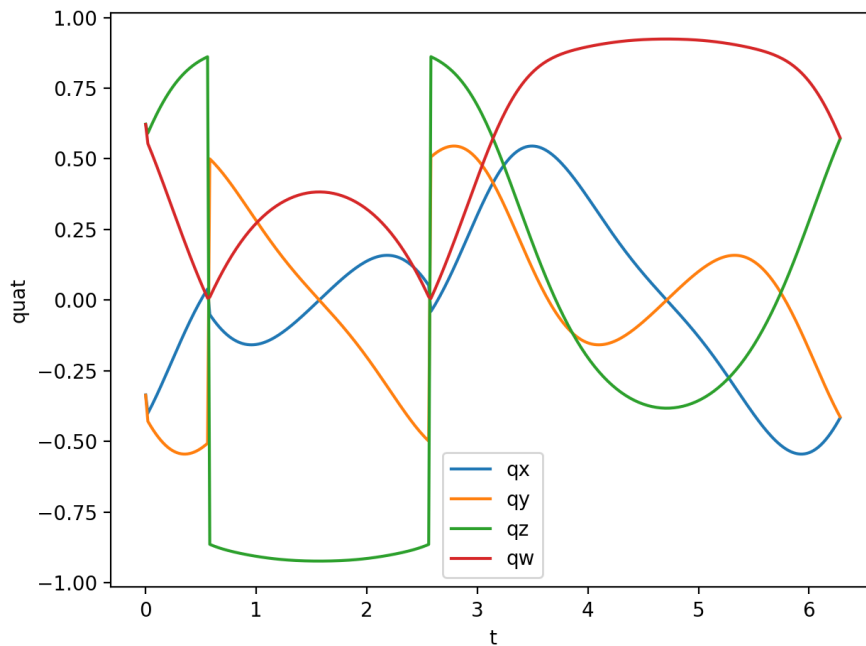


图 2: Homework 题目 3

2 Code 部分

2.1 任务一：补全四旋翼动力学模型

2.1.1 题目描述

- 在指定文件 `Quadrotor.cpp` 中补全四旋翼的动力学方程，使仿真器能够正确计算状态导数并进行数值积分。
- 需要补全的核心内容包括：
 - 线速度导数 $\dot{\mathbf{v}}$ ：应与重力、总推力、外力以及空气阻力等因素相关；
 - 角速度导数 $\dot{\boldsymbol{\omega}}$ ：涉及惯性矩阵 $\mathbf{J}$ 的逆、力矩、由 $\boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})$ 表示的科里奥利/陀螺项以及外部力矩等。
- 完成后需通过编译（`catkin_make`）并运行悬停测试（`test_control.launch`）验证模型补全正确；报告中需简洁说明补全思路，避免粘贴大段代码。

2.1.2 补全思路

四旋翼动力学可分为平动与转动两部分。仿真器的状态通常包含位置 $\mathbf{x}$ 、速度 $\mathbf{v}$ 、姿态（旋转矩阵 $\mathbf{R}$ 或四元数）以及机体系角速度 $\boldsymbol{\omega}$ 。在每个仿真步内，根据当前状态与输入（总推力、力矩等）计算状态导数并交由数值积分器更新。

1) 平动动力学：求 $\dot{\mathbf{x}}$ 与 $\dot{\mathbf{v}}$ 平动部分满足

$$\dot{\mathbf{x}} = \mathbf{v}.$$

线加速度 $\dot{\mathbf{v}}$ 来自合力除以质量：

$$m\dot{\mathbf{v}} = m\mathbf{g} + \mathbf{F}_{\text{thrust}} + \mathbf{F}_{\text{ext}} + \mathbf{F}_{\text{drag}}.$$

其中：

- 重力项取世界系常量 $\mathbf{g} = g\mathbf{e}_3$ （方向由仿真器坐标系约定确定）；
- 总推力在机体系通常沿机体 z 轴方向（即 $\mathbf{e}_3^B$ ），需由姿态旋转到世界系：

$$\mathbf{F}_{\text{thrust}} = \mathbf{R}(0, 0, T)^\top,$$

其中 T 为总推力标量（或由四桨推力合成得到）；

- 外力 $\mathbf{F}_{\text{ext}}$ 由接口/传感器或环境模块给出（若无则为零）；

- 空气阻力采用与速度成正比的线性阻尼：

$$\mathbf{F}_{\text{drag}} = -k_d \mathbf{v} \quad (\text{或按轴分别阻尼}).$$

最终得到

$$\dot{\mathbf{v}} = \mathbf{g} + \frac{1}{m} \mathbf{R}(0, 0, T)^\top + \frac{1}{m} \mathbf{F}_{\text{ext}} - \frac{k_d}{m} \mathbf{v}.$$

2) 转动动力学：求 $\dot{\omega}$ 转动部分使用刚体欧拉方程（角速度在机体系表达）：

$$\mathbf{J} \dot{\omega} = \boldsymbol{\tau} - \omega \times (\mathbf{J} \omega) + \boldsymbol{\tau}_{\text{ext}},$$

其中 $\mathbf{J}$ 为惯性矩阵， $\boldsymbol{\tau}$ 为控制力矩输入， $\boldsymbol{\tau}_{\text{ext}}$ 为外部扰动力矩（若无则为零）。因此

$$\dot{\omega} = \mathbf{J}^{-1}(\boldsymbol{\tau} - \omega \times (\mathbf{J} \omega) + \boldsymbol{\tau}_{\text{ext}}).$$

同时，姿态导数由仿真器既定表示法决定（若内部用四元数，则按 $\dot{\mathbf{q}} = \frac{1}{2}(\omega)\mathbf{q}$ ；若用旋转矩阵，则用 $\dot{\mathbf{R}} = \mathbf{R}[\omega]_\times$ ），本任务只需保证 $\dot{\omega}$ 与合力/合力矩一致即可。

2.1.3 结果展示

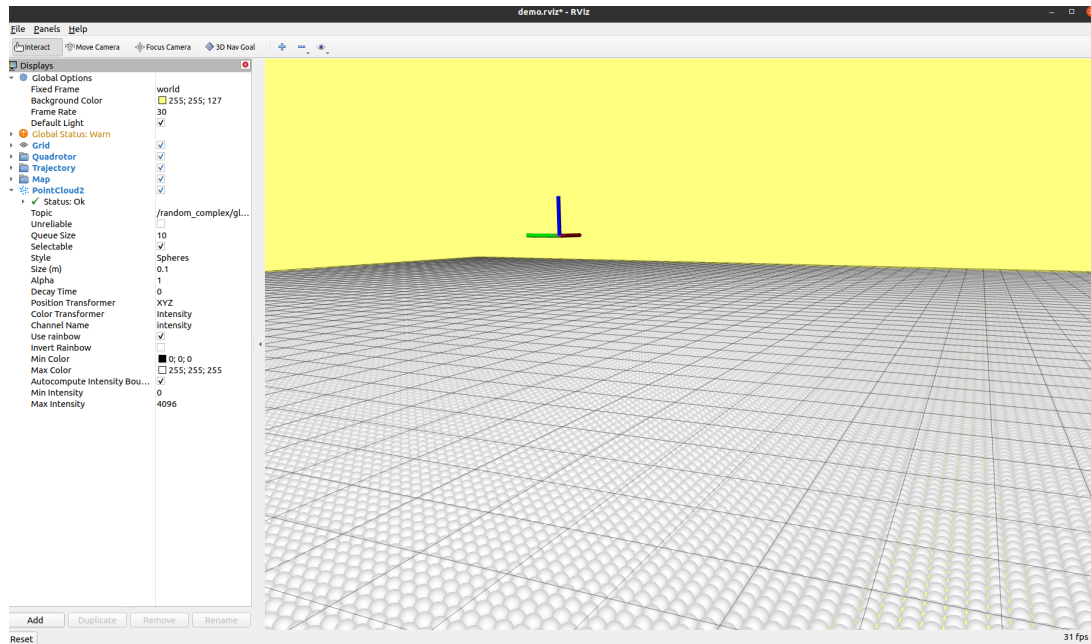


图 3: 任务一悬停测试

2.2 任务二：补充前端规划模块（A* 搜索）

2.2.1 题目描述

- 在 `Astar_searcher.cpp` 中补全 A* 路径搜索的关键步骤，使无人机能够在点云/栅格地图中规划到目标点的无碰撞路径。

- 需要完成的内容包括：
 - **STEP 1.1:** 补全启发式函数 `getHeu` (距离度量、启发式形式, 以及是否使用 `tie_breaker`);
 - **STEP 1.2:** 补全 A* 主循环的关键逻辑 (弹出 $f = g + h$ 最小节点、终点判定、邻居扩展、信息更新等);
 - **STEP 1.3:** 追溯父节点生成最终路径并返回。
- 完成后需编译运行 `demo.launch`, 并在 RViz 中使用 3D Nav Goal 设置目标点验证规划可用; 报告中需简洁说明补全思路, 避免粘贴大段代码。

2.2.2 补全思路

本任务的目标是在已给定的栅格地图与节点数据结构基础上, 实现标准 A* 搜索流程, 并输出一条由栅格中心点组成的无碰撞路径。整体思路是: 用启发式函数指导搜索方向 (*STEP 1.1*), 用 *open/closed* 集合推进搜索 (*STEP 1.2*), 最终通过父指针回溯得到路径 (*STEP 1.3*)。

STEP 1.1: 启发式函数 `getHeu` 启发式函数 $h(\cdot)$ 用于估计当前节点到目标节点的代价, 满足尽量低估真实代价以保证 A* 的最优性。我们按常规做法采用欧氏距离 (对应 26 邻接栅格的自然距离度量):

$$h(\mathbf{n}) = \|\mathbf{p}(\mathbf{n}) - \mathbf{p}(\mathbf{n}_{goal})\|_2,$$

并可选地加入很小的 `tie_breaker` (例如 $1 + \epsilon$ 的系数) 来缓解大量等 f 值导致的搜索抖动, 但不改变主要代价趋势。该函数只负责返回一个标量启发值, 最终 $f = g + h$ 。

STEP 1.2: A* 主循环 A* 的核心是维护一个按 $f = g + h$ 排序的 open set (优先队列或可排序容器):

- 初始化: 把起点节点压入 open set, 设置其 $g = 0$, $h = \text{getHeu}(\cdot)$, 并记录父节点为空;
- 循环: 每次从 open set 取出 f 最小的节点作为当前节点;
- 终止: 若当前节点到达目标 (索引相同或距离小于阈值), 记录终点指针并结束;
- 扩展邻居: 遍历当前节点的邻居 (通常为 26 邻域), 对每个邻居做:
 1. 可行性检查: 是否在地图边界内、是否为障碍栅格 (占据/碰撞), 不可行则跳过;

2. 代价更新：计算从当前到邻居的边代价（与邻接方向长度一致），得到候选

$$g_{\text{new}} = g_{\text{cur}} + \text{cost}(\text{cur} \rightarrow \text{nbr});$$

3. 若邻居未被访问过：写入其 $g/h/f$ 与父指针，加入 open set；
4. 若邻居已在 open set 且 g_{new} 更小：更新其 g/f 并重设父指针（必要时更新其在优先队列中的位置/重新入队）。

实现上我依赖代码中已有的节点状态标记（如 OPENSET/CLOSEDSET/UNVISITED），并在出队后把当前节点标记为 closed，避免重复扩展。

STEP 1.3: 路径回溯与输出 当找到终点后，通过终点节点的父指针链回溯到起点，得到一串离散栅格节点序列。为得到可用于后续轨迹生成的几何路径，我将每个节点的栅格索引转换为世界坐标（栅格中心）并反转顺序，形成从起点到终点的路径：

$$\mathbf{p}_0 \rightarrow \mathbf{p}_1 \rightarrow \cdots \rightarrow \mathbf{p}_N.$$

同时，为 RViz 可视化需要，还会输出搜索过程中访问过的节点集合 (`getVisitedNodes`)，用于检查搜索范围是否合理。

2.2.3 结果展示

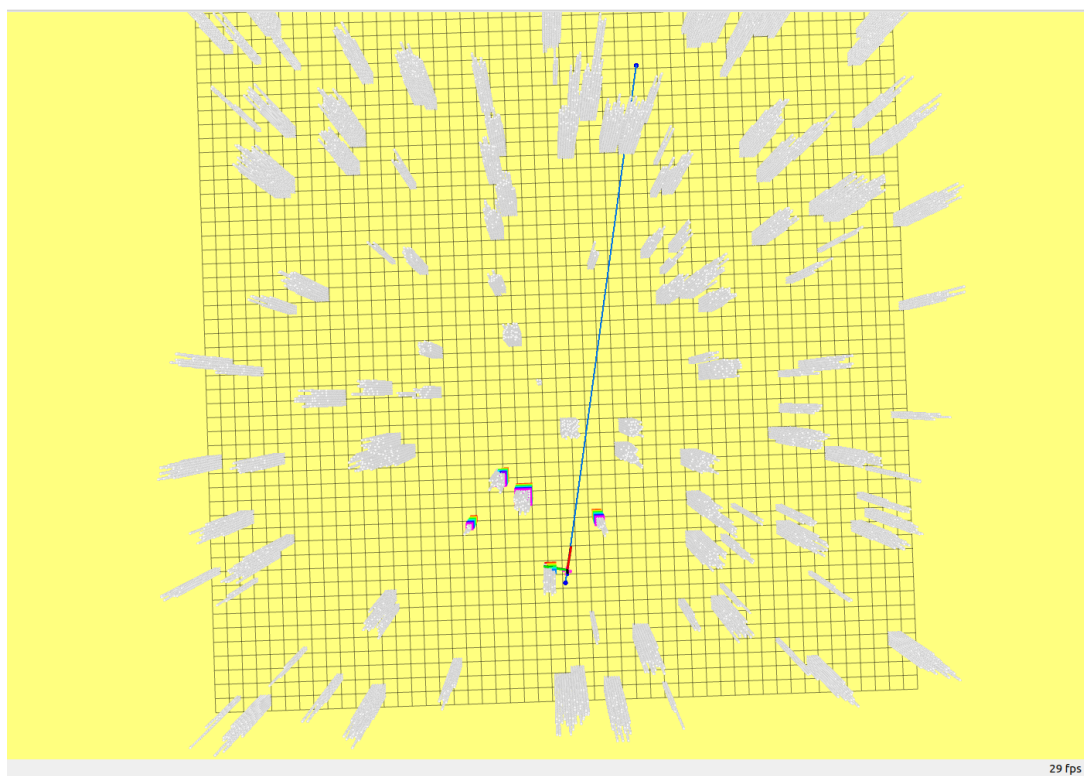


图 4: 任务二路径规划测试

2.3 任务三：控制器参数调试与评估

2.3.1 题目描述

- 在完成动力学与规划模块后，对 SO(3) 位置/速度控制器的增益参数进行调试，使无人机对轨迹的跟踪更稳定、误差更小。
- 具体要求：阅读 `so3_control_nodelet.cpp` 中控制器实现，调整 k_x 与 k_v 参数。
- 评估与提交：运行 `calculate_results.py` 得到 RMSE、轨迹运行时间、总长度、碰撞状态与综合得分等指标；将终端输出粘贴进报告并进行必要分析（包括调参思路与技巧）。

2.3.2 调参思路与初步结果

本任务仅对 SO(3) 位置/速度控制器的增益 k_x 与 k_v 进行调试，保持动力学模型、规划前端与控制器结构不变。调参目标是在不发生碰撞的前提下尽量降低轨迹跟踪误差，并兼顾运行时间与轨迹长度对综合分数的影响。评估方式为：每次修改参数后运行仿真并执行 `calculate_results.py`，记录 RMSE、总时间、总长度与综合得分，形成对比表后选择最优组合。

控制规律中 k_x 主要作用于位置误差项（增强“拉回轨迹”的能力）， k_v 主要作用于速度误差项（提供阻尼抑制振荡）。因此参数变化的典型现象为： k_x 过大容易造成超调与机身抖动， k_v 过小则阻尼不足导致摆动；而 k_v 过大又会使系统响应变慢、出现明显滞后，从而增大跟踪误差。基于上述特性，采用如下的调参流程：

- 先保证稳定性：**以中等 k_x 起步，优先调大 k_v 使飞行过程中无明显振荡与抖动，确保轨迹能够完整跑通且无碰撞；
- 再降低误差：**在稳定的前提下，小幅提高 k_x 以增强跟踪能力；若出现超调或抖动，则相应提高 k_v 或回退 k_x ；
- 按轴微调：**结合轨迹特性与坐标系分量差异，保持 z 轴的 k_x 略高以抵抗重力与垂向耦合；对更易出现侧向摆动的轴适度提高 k_v 增强阻尼；
- 小步迭代与记录：**每次仅改动少量参数，避免多因素同时变化造成结论不清；用脚本输出对比 RMSE 与综合得分，最终以综合得分最低者为准。

通过上述迭代，最终在 $k_x = (8.0, 8.0, 7.0)$ 、 $k_v = (4.5, 7.0, 5.0)$ 时取得最佳结果，综合分数约为 30.9。该组合在位置跟踪能力与阻尼抑振之间取得较好平衡：飞行过程稳定、抖动较小，且整体 RMSE 相对更低。也为接下来的继续优化打好了基础。

```

stuwork@ubuntu:~$ python3 /home/stuwork/MRPC-2025-homework/MRPC-2025-homework/co
de/calculate_results.py
计算得到的均方根误差 (RMSE) 值为: 0.041939796812969804
轨迹运行总时间为: 79.70083284378052
总轨迹长度为: 33.14092624409242
是否发生了碰撞: 0
综合评价得分为(综合分数越低越好): 30.956311180168548
stuwork@ubuntu:~$

```

图 5: 任务三初步结果

2.4 任务四（附加题）：有效改进与对比实验

2.4.1 题目描述

- 本任务为选做附加题，目标是在 baseline 的规划与控制框架上进行**有效改进**，并通过对比实验展示改进带来的性能提升。
- 可选改进方向包括但不限于：
 - **前端规划改进**：例如 JPS3D、Hybrid A*、RRT 等；
 - **后端轨迹优化**：修改/替换轨迹优化算法；
 - **控制器改进**：编写新的控制器架构，例如将姿态环与角速度环从 `so3_quadrotor_simulator` 移植到 `so3_control`。
- 报告要求：清晰说明改进动机、方法与工程改动点，并进行必要的对比实验；评分重点在新颖性与最终飞行效果。

2.4.2 改进思路与对比实验

本附加题在任务三的 baseline (A* 前端 + 原后端轨迹生成 + 仅调 k_x, k_v 的位置/速度控制) 基础上，做了三类优化：**A* 路径 shortcut 后处理**、**控制器输出力矩的 SO(3) 姿态/角速度闭环**、以及**时间分配 (time allocation) 微调**。三者分别对应“让参考轨迹更易跟踪”“让姿态响应更可控”“让局部动态需求更合理”，共同降低跟踪误差与综合得分。

改进 1：前端路径 shortcut (减少冗余拐点)

动机：A* 的栅格路径天然呈折线，障碍物附近拐点多、局部曲率大；后端即使生成可行轨迹，也会因为参考路径“拐得太急”导致控制器频繁修正，RMSE 增大、姿态抖动明显。

代码实现：在前端 A* 得到离散路径后，调用 `Astarpath::shortcutPath(...)` 对路径做“可视直连”简化：对一段路径尝试用直线替代，若直线段通过地图的碰撞检测（沿线采样检查占据）则删除中间节点，保留关键转折点。该逻辑位于 `Astar_searcher.cpp` 的 `shortcut` 相关函数中，入口在轨迹生成节点拿到 `grid_path` 后进行后处理。

效果分析： shortcut 直接减少路径点数量与多余拐点，使后端生成的多项式段数与曲率变化降低；从 RViz 观察可见障碍密集区域的蓝色轨迹更直更简洁（如图 6 所示），从而降低控制器的瞬时加速度/姿态变化压力。

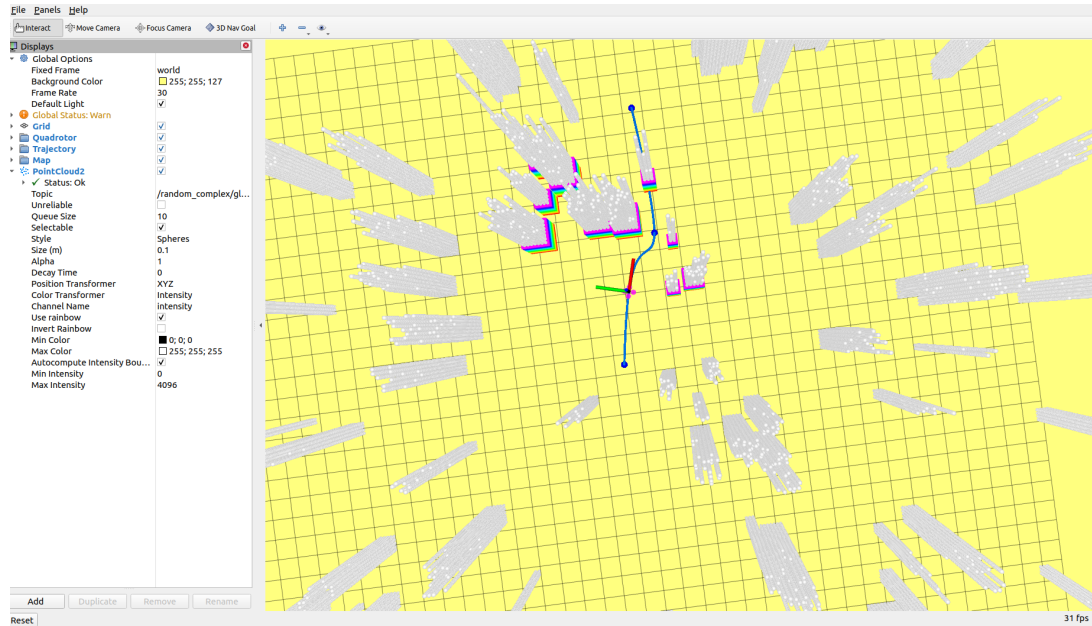


图 6: 障碍密集区轨迹示意

改进 2: 控制器改进（显式输出力矩，闭合姿态/角速度环）

动机： baseline 中位置环输出推力方向，但姿态与角速度环的闭环在仿真器内部完成，工程上不利于针对急转/密集障碍场景做增强；且当参考轨迹变化较快时，姿态环响应不足会表现为“机身跟不上轨迹、误差被放大”。

代码实现：

- S03Command.msg 增加力矩字段（例如 moment），使控制器可以把力矩命令随同姿态/推力一起发布；
- so3_control/S03Control.h, S03Control.cpp 在 calculateControl(...) 内基于当前姿态 $\mathbf{R}$ 与期望姿态 $\mathbf{R}_d$ 计算 SO(3) 姿态误差，并结合角速度误差生成力矩：

$$\mathbf{e}_R = \frac{1}{2} \text{vee}(\mathbf{R}_d^\top \mathbf{R} - \mathbf{R}^\top \mathbf{R}_d), \quad \mathbf{M} = -\mathbf{K}_R \mathbf{e}_R - \mathbf{K}_\Omega \mathbf{e}_\Omega + \boldsymbol{\Omega} \times (\mathbf{J} \boldsymbol{\Omega}),$$

同时增加 setOrientation(...), setOmega(...) 等接口以接入里程计/IMU；

- so3_control_nodelet.cpp 从 odom/IMU 更新当前姿态与角速度，调用 calculateControl 后把推力与力矩写入 S03Command 发布；
- so3_quadrotor_simulator/quadrotor_simulator_so3.cpp 的回调中读取 S03Command 的力矩字段，直接作为刚体转动方程的控制输入，完成从控制器 → 仿真器的力矩闭环。

效果分析：该改动使姿态/角速度闭环由 `so3_control` 显式实现并可控，急转/密集障碍段的姿态响应更稳定，机身抖动减少；从评估上主要体现为 RMSE 的降低与轨迹跟随更贴近参考。

改进 3：时间分配微调（降低局部动态需求）

动机：在障碍物密集区域，路径段往往更短、转角更大。若时间分配仍按统一速度/加速度假设给出较短段时，会造成参考轨迹在短时间内需要更高加速度/姿态变化率，控制器更难跟上，从而出现 RMSE 突增甚至碰撞风险。

代码实现：在 `trajectory_generator_node.cpp` 的 `timeAllocation(...)` 中，保留原“按段距离分配时间”的主框架，仅对**短段/急转段**施加额外的时间放大系数：例如对相邻段夹角较大、或段长接近地图分辨率的段，适当增加 T_i ，相当于在这些位置“自动降速”。

效果分析：该策略不改变轨迹形状，但降低了难段的动态需求，使无人机在密集障碍区域的跟踪更平滑，RMSE 峰值更低；直线长段则基本不受影响，从而避免整体时间被过度拉长。

对比实验设置与结果 为保证对比公平，实验中保持地图、目标设置、launch 配置与评估脚本一致，并固定与任务三相同的控制增益：

$$k_x = (8.0, 8.0, 7.0), \quad k_v = (4.5, 7.0, 5.0).$$

- **baseline（任务三）：**仅调 k_x, k_v ，综合得分约为 30.9；
- **附加题（加入 shortcut + 控制器改进 + time allocation 微调）：**在相同 k_x, k_v 下综合得分下降至约 20.7。（如下图所示）

```
stuwork@ubuntu:~$ python3 /home/stuwork/MRPC-2025-homework/MRPC-2025-homework/co
de/calculate_results.py
计算得到的均方根误差（RMSE）值为：0.0315548330963019
轨迹运行总时间为：43.1103413105011
总轨迹长度为：29.15944702971271
是否发生了碰撞：0
综合评价得分为(综合分数越低越好)：20.764924287303142
```

图 7: 改进后与任务三对比

从仿真结果上看，改进后在障碍密集段的参考轨迹更简洁（shortcut 减少冗余拐点），姿态控制更稳定（显式力矩闭环抑制抖动），并且在急转与短段处跟踪不再“硬追”（时间分配增加局部裕度），因此 RMSE 与综合得分均出现明显改善。

显然上述结果仍并非最优，在进一步按照任务三的调参步骤调整后，最终得到了最好的结果。

2.4.3 最终结果展示

最终，我们在

$$k_x = (9.5, 9.5, 7.3), \quad k_v = (5.0, 7.5, 5.0)$$

下取得了综合表现最优的一组结果。评估脚本输出为：RMSE = 0.0142，轨迹总时间 = 34.11 s，总长度 = 30.00 m，碰撞次数 = 0，综合得分 = 15.66（综合分数越低越好）。对应的仿真轨迹与可视化结果如图 8 所示。

```
stuwork@ubuntu:~$ python3 /home/stuwork/MRPC-2025-homework/MRPC-2025-homework/co
de/calculate_results.py
计算得到的均方根误差 (RMSE) 值为: 0.014180645331135172
轨迹运行总时间为: 34.10915660858154
总轨迹长度为: 30.00405838297496
是否发生了碰撞: 0
综合评价得分为(综合分数越低越好): 15.658772064538336
```

图 8: 最终结果

从指标上看，该结果具有以下特点：

- **跟踪精度高：** RMSE 降至 1.4×10^{-2} 量级，表明无人机在全程对参考轨迹的偏差较小，尤其在转弯与障碍物密集区域仍能保持较好的贴合度；
- **效率与可行性兼顾：** 总时间约 34 s，总长度约 30 m，说明规划与轨迹生成并未因追求平滑而显著绕行或过度降速；
- **安全性满足要求：** 碰撞为 0，说明轨迹在地图约束下保持了可行与安全。

本组增益在“跟踪能力”和“阻尼抑振”之间取得了较好平衡：适度提高 k_x 增强了位置误差纠正能力，而相对较大的 k_v 为系统提供了足够的阻尼，抑制了超调与振荡，使得误差不会在复杂路段被放大。

该组参数能够复现较低的 RMSE，并在时间与长度指标上维持较优水平，因此最终选为提交结果。